ATC Readback Verifier

An NLP second-check on the pilot read-back loop

NLP Group Project: Option 1 (Application Development)

Team: Jan · Vlad · Felipe · Alberto · Kishan · Yi

Repo: github.com/janwejchert/NLP-project · Live demo: nlp-project-atc.streamlit.app

> A proof of concept, not production / operational software.

The problem: read-back safety

- A controller issues an **instruction**; the pilot **reads it back**; the controller confirms it (the *hear-back*). This loop is the main defence against miscommunication.
- It depends entirely on a human catching an error in real time.
- Documented safety research (NASA ASRS; FAA/Volpe) finds controllers catch only roughly **two-thirds** of incorrect read-backs, about a third go undetected.
- Error likelihood rises with message complexity; headings, frequencies and altitudes are among the most error-prone items.

An uncorrected read-back error can stay hidden until a loss of separation appears.

A concrete example

Correct read-back → MATCH

```
Instruction: Speedbird 245, descend flight level 240.
Readback:    Descend flight level 240, Speedbird 245.
Verdict:     MATCH: readback is correct.
```

Wrong altitude → DISCREPANCY

```
Instruction: Speedbird 245, descend flight level 240.
Readback:    Descend flight level 250, Speedbird 245.
Verdict:     DISCREPANCY: instructed FL240, read back as FL250
             [value_substitution]
```

A different value, a transposed digit, an omitted item: small slips, real consequences.

The gap in existing tools

- Most mature / commercial ATC speech systems **transcribe** audio; they do not **verify** that a read-back is correct (e.g. Appareo's ASR system advertises no clearance verification).
- The ATCO2 corpus and MIT/MITRE work focus on transcription and concept extraction: building blocks, not a fielded verifier.
- Dedicated read-back-error-detection prototypes exist mostly in European SESAR-funded research (e.g. HAAWAII), reporting ~80–81% detection, still lab-stage, with non-trivial false-alarm rates.
- ICAO doctrine **requires** the check but leaves it a manual human task.

Our niche: a lightweight, auditable, text-based field-by-field verifier with an explicit error taxonomy.

Our approach: a hybrid pipeline

The LLM does **only** extraction. **Every** judgement is deterministic Python: our core IP.

```
    instruction text            read-back text
         │                            │
         ▼                            ▼
    [ LLM Extractor ]          [ LLM Extractor ]   swappable backend
         │  8 fields                  │  8 fields
         └─────────────┬──────────────┘
                       ▼
           [ Deterministic Comparator ]   field-by-field, normalized
                       │  list of discrepancies      (no LLM, unit-tested)
                       ▼
                [  Verdict  ]   MATCH, or DISCREPANCY + specific problems
```

This separates fallible language understanding from the comparison that actually flags errors, keeping the verification logic **auditable and reproducible**.

What we extract and what we judge

Eight structured fields (the ICAO mandatory read-back items):

`callsign` · `altitude` · `heading` · `speed` · `frequency` · `squawk` · `runway` · `qnh`

Five error categories:

Category	Meaning
`value_substitution`	a different value was read back
`digit_transposition`	same digits, reordered (e.g. runway 21 → 12)
`omission`	an instructed item was not read back
`callsign_error`	callsign wrong or missing
`added_element`	read-back contains an item not instructed

A correct read-back = zero discrepancies = **MATCH**.

Live demo

The Streamlit web app

- Type the controller **instruction** and the pilot **read-back** as text.
- Get a verdict: **MATCH**, or **DISCREPANCY** with the specific problems listed.
- Hosted on Streamlit Community Cloud (extraction backend = Hugging Face / Qwen2.5-7B): **nlp-project-atc.streamlit.app**

[Switch to the app, run the MATCH example, then introduce a wrong digit and a missing item.]

> Scope: text input only. Live audio / speech-to-text is future work, not built here.

Two free backends

Selected by the `EXTRACTOR_BACKEND` env var, both $0.

Backend	Runs	Default model	Used for
`ollama`	Locally, offline	`qwen2.5:3b`	Development + the **reproducible eval**
`hf`	HF Inference API (free token)	`Qwen2.5-7B-Instruct`	The hosted Streamlit demo

- Same `ExtractedFields` object from both, so everything downstream is backend-agnostic.
- The cloud free tier cannot host a local model → demo uses `hf`; **graded metrics come from the local `ollama` run.**

Evaluation method

- Our **own 50-case labelled test set** (`TC01` .. `TC50`), covering MATCH plus every error category.
- Positive class = **"read-back contains an error"**, so precision / recall / F1 measure error **detection**.
- **False-alarm rate** = fraction of correct read-backs wrongly flagged, the key metric in a safety setting (false alarms erode trust).
- Also reported: verdict accuracy, per-category detection recall, and a confusion matrix.
- Temperature 0, pinned model+tag, committed results → reproducible with `make eval`.

Headline results

Local `ollama` / `qwen2.5:3b`, 50 cases (final):

Precision	Recall	F1	False-alarm	Verdict accuracy
0.971	1.000	0.986	6.2%	0.980

- **100% detection recall in every one of the five categories.**
- Confusion matrix: 15 TN, 1 FP, 0 FN, 34 TP.
- Just **1 remaining verdict failure** (TC12, an extraction miss, not a comparator error).

The iteration story (critical thinking)

The journey mattered more than the final number.

Stage	Prompt	F1	False-alarm
Baseline	3b, original	0.957	12.5%
Hardened	+rules +examples	**0.944**	**25.0%** ⚠️
Final	safe rules + comparator rule	**0.986**	**6.2%**

- A naive prompt-**hardening** attempt **regressed**, false-alarm rate *doubled*.
- Over-specific few-shot examples **poisoned the small model**: it read the 4-digit callsign suffix `Easy 4471` as a squawk, and misread `runway 24` as `heading 240`.
- The fix: a **deterministic comparator rule for runway side** (match number; compare side only when both messages state one) + **safe prompt rules**, not more examples.

Conclusion: the comparator was never at fault; extraction is the bottleneck; few-shot examples can backfire on small models.

Failure analysis: the one remaining miss

TC12: a genuine small-model limitation

```
Instruction: Easy 4471, reduce speed 210 knots, descend flight level 100.
Readback:    Speed 210 knots, descend flight level 100, Easy 4471.
Gold: MATCH   Predicted: DISCREPANCY (added_element, altitude)
```

- The 3B model extracted `FL100` from the **read-back** but **dropped it from the instruction**, inconsistent extraction on a two-clause message → a spurious added element.
- Not a logic error: the comparator is unit-tested and never caused a failure.
- The kind of miss a **larger model** is expected to fix.

3B vs 7B

- The same prompt and comparator were also run on **`qwen2.5:7b`**, to test whether more model capacity resolves the residual extraction errors (like TC12).
- The comparison table and chart live in **`notebooks/analysis.ipynb`** (section 4) and `eval/results/runs/7b_final_v3/`.

> See the notebook for the side-by-side, we let the data speak rather than quoting a single headline number.

Limitations & future work

Limitations (it's a POC):

- Text input only: no speech-to-text / live radio audio.
- Not production-grade: no auth, no database, no multi-user state.
- Off-the-shelf instruction models with prompting; no fine-tuning.
- Residual extraction misses on complex multi-clause messages.

Future directions:

- Whisper-style speech-to-text front end for live audio.
- Phonetic / NATO-alphabet robustness; accent and noise handling.
- Multi-model ablation (local vs hosted) and confidence scoring.
- Controller-facing alerting UX.

What we learned

- **Separating extraction from judgement** makes the system auditable, reproducible, and defensible, and lets a small free model suffice.
- **Measuring made it possible:** a reproducible harness turned "the prompt feels better" into a measured −18.8 pp false-alarm swing between attempts.
- **Few-shot examples can poison a small model**; safe rules and deterministic post-processing were more reliable levers.
- **The deterministic comparator was never the bottleneck**, extraction was.

Use of AI tools: AI assisted with scaffolding, prompt drafting, prose drafts, and literature search. Every substantive decision, problem framing, test-set labels, error taxonomy, comparison logic, methodology, is the team's own and defensible in Q&A. (Full section in `docs/report/use_of_ai_tools.md`.)

Thank you, questions?

ATC Readback Verifier · Jan · Vlad · Felipe · Alberto · Kishan · Yi

- Repo: github.com/janwejchert/NLP-project
- Live demo: Streamlit Community Cloud
- Field review, failure analysis, metrics & notebook in `docs/report/` and `eval/results/`

Headline: F1 0.986 · false-alarm 6.2% · 100% detection recall in every category.

Q&A